

UTILISATION DE RÉSEAUX EXISTANTS

SOMMAIRE

1-	Quelques réseaux profonds classiques	1
	1.1- AlexNet	
	1.2- VGG	
	1.3- Inception	
	1.4- ResNet	
	1.5- SqueezeNet	
2-	Comment utiliser ces réseaux	6
	2.1- Utilisation de réseaux pré-entraînés	
	2.2- Transfer learning et fine tuning	
3-	Que faire si j'ai peu de données ?	7
4-	Partie pratique	9

Depuis la fin des années 90, de nombreux réseaux profonds ont vu le jour et se sont complexifié, diversifié, pour répondre à des problèmes de plus en plus vastes. [1] propose une analyse comparative de ces réseaux et décrit en particulier leurs performances en fonction du nombre d'opérations (figure 0-1).

Nous présentons dans la suite cinq réseaux profonds classiques. Nous montrons ensuite comment les utiliser directement, ou comment les adapter pour répondre à une problématique précise, en lien avec leur utilisation originale ou non. Nous introduisons enfin une manière d'apprendre un réseau à partir de peu de données.

1- QUELQUES RÉSEAUX PROFONDS CLASSIQUES

Les cinq réseaux présentés ici ont prouvé leur efficacité, notamment lors des compétitions organisées depuis 2010 sur une base de données d'images nommée [ImageNet](#). Initiée à l'Université de Stanford, cette base de données comporte aujourd'hui plus de 14 millions d'images, classées en 21841 catégories (avions, voitures, chats,...). Dans les compétitions ILSVRC (ImageNet Large Scale Visual Recognition Challenge), les chercheurs se voient proposer une extraction de 1,2 millions d'images, catégorisées en 1000 classes, et le gagnant est celui qui atteint la meilleure précision de reconnaissance sur les 5 premières classes (top-5).

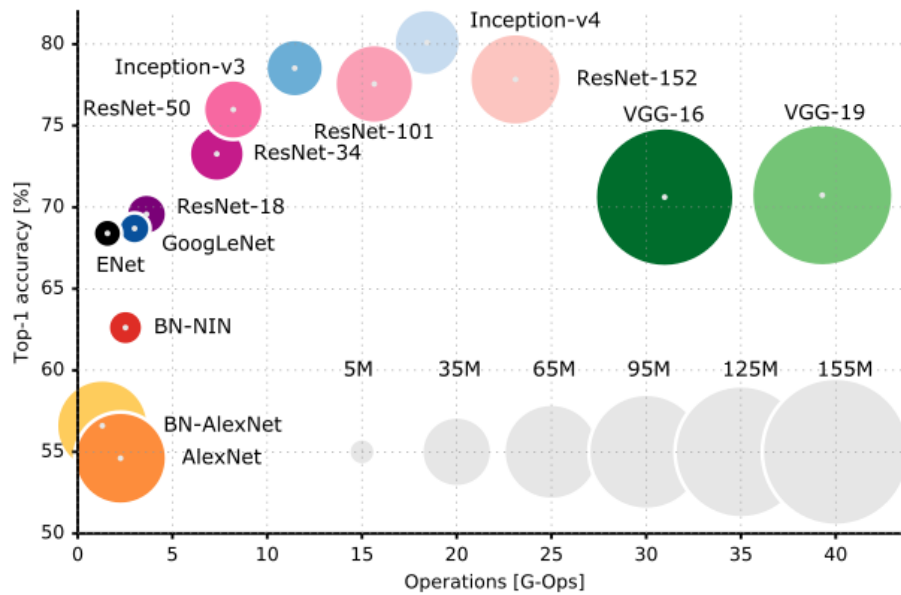


FIGURE 0-1 – Précision en fonction du nombre d'opérations nécessaire pour un calcul en passe avant. La taille des blobs est proportionnelle au nombre de paramètres du réseau (source : [1])

1.1- AlexNet

En 2012, Krizhevsky et al [5] remportent ILSVRC avec un taux de reconnaissance de 84.6%, en utilisant AlexNet, un réseau convolutif composé de 5 couches de convolution et de pooling, suivies de 3 couches complètement connectées (figure 1-2).

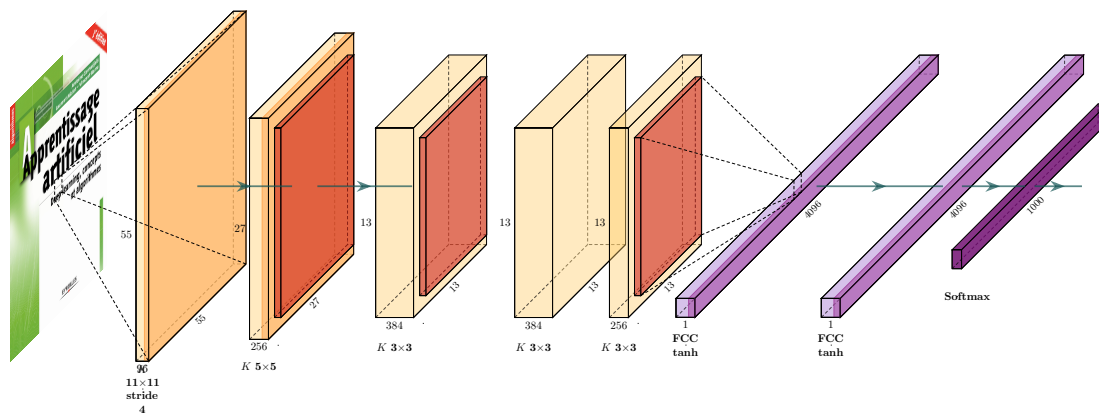


FIGURE 1-2 – Architecture du réseau AlexNet. Les couches de convolution et d'activation sont en orange clair, les couches d'agrégation en orange foncé. Les couches complètement connectées sont en violet.

Si la profondeur du réseau reste faible, le nombre de paramètres était déjà important. En regardant uniquement la première couche de convolution, on constate que :

- l'entrée est composée d'images $227 \times 227 \times 3$
- les filtres de convolution sont de taille 11
- le pas de convolution (stride) est de 4

Ainsi la sortie de la couche de convolution est de taille $55 \times 55 \times 96 = 290\,400$ neurones, chacun ayant $11 \times 11 \times 3 = 363$ poids et un biais. Cela implique, sur cette couche de convolution seulement, 105 705 600 paramètres à ajuster.

Ce réseau, amélioration d'un réseau existant (LeNet), apportait de nombreuses contributions, comme l'utilisation de couches ReLU, de dropout, ou du GPU (NVIDIA GTX 580) pendant la phase d'entraînement.

1.2- VGG

Les réseaux VGG (Visual Geometry Group, université d'Oxford) [8] ont été les premiers réseaux à utiliser de petits filtres de convolution (3×3) et à les combiner pour décrire des séquences de convolution, l'idée étant d'émuler l'effet de larges champs réceptifs par cette séquence. Cette technique amène malheureusement à un nombre exponentiel de paramètres (le modèle entraîné qui peut être téléchargé a une taille de plus de 500 Mo). VGG a concouru à ILSVRC 2014, a obtenu un taux de bonne classification de 92.3% mais n'a pas remporté le challenge. Aujourd'hui VGG et une famille de réseaux profonds (de A à E) qui varient par leur architecture (figures 1-3 et 1-4). Le nombre de paramètres (en millions) pour les réseaux de A à E est 133, 133, 134, 138 et 144. Les réseaux VGG-D et VGG-E sont les plus précis et populaires.

A	A-LRN	B	C	D	E
11 couches	11 couches	13 couches	16 couches	16 couches	19 couches
Entrée : image 224×224 RGB					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
max pooling					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
max pooling					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
max pooling					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
max pooling					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
max pooling					
Couche complètement connectée 4096 neurones					
Couche complètement connectée 4096 neurones					
Couche complètement connectée 1000 neurones					
Classifieur softmax					

FIGURE 1-3 – Architectures des réseaux VGG

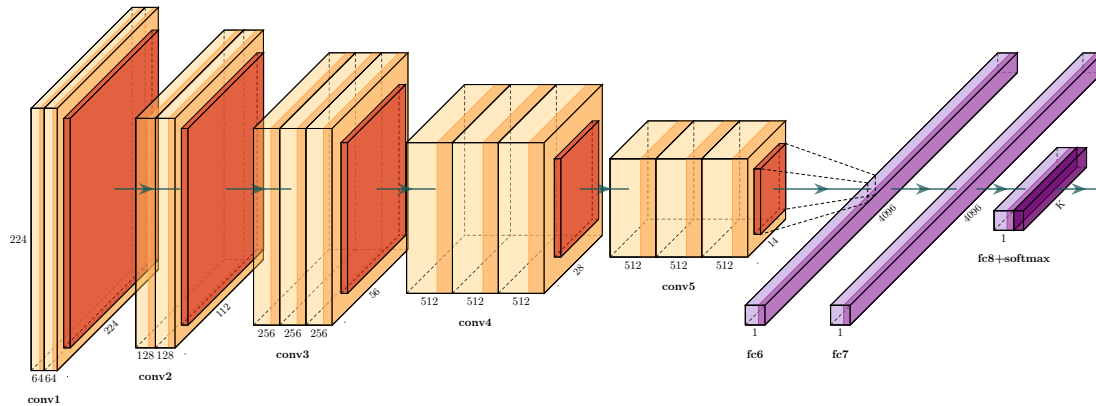


FIGURE 1-4 – Réseau VGG16

1.3- Inception

Inception, proposé par Google, est le premier réseau dont les performances ont été augmentées pas seulement en augmentant le nombre de couches, mais en pensant et optimisant le design et l'architecture. L'idée est ici d'utiliser plusieurs filtres, de tailles différentes, sur la même image et de concaténer les résultats pour générer une représentation plus robuste.

Inception n'est pas un réseau, c'est une famille de réseaux : Network in Network [6], Inception V1 [10], Inception V2 [11], Xception [2],...

L'idée du premier réseau (figure 1-5) est de connecter les couches de convolution par des perceptrons multicouches, introduisant des non linéarités dans les réseaux profonds. Mathématiquement, ces perceptrons sont équivalents à des convolutions par des filtres 1×1 et gardent donc la cohérence des réseaux. Cette nouvelle architecture rend moins indispensable les couches complètement connectées en fin de réseau. Les auteurs moyennent spatialement les cartes finales et donnent le résultat au classifieur softmax. Le nombre de paramètres est alors réduit, diminuant de ce fait le risque de sur apprentissage.

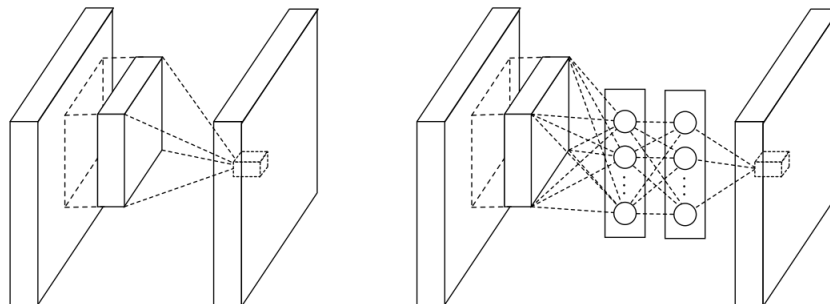


FIGURE 1-5 – Réseau Network in Network

Inception V1, implémenté dans le réseau GoogLeNet vainqueur d'ILSVRC 2014, est une extension à des réseaux plus profonds de Network to Network. Le réseau est composé de 22 couches et atteint 93.3% de taux de reconnaissance. D'autres améliorations théoriques (fonctions de pertes associées aux couches intermédiaires dans la phase d'apprentissage, introduction de caractères épars dans le réseau) ont également permis d'améliorer les performances (de calcul et de classification).

Inception V2, puis V3 (figure 1-6) adoptent des techniques de factorisation (toute convolution par un filtre de taille plus grande que 3×3 peut être exprimée de manière plus efficace avec une série de filtres de taille réduite) et de normalisation pour améliorer encore les performances.

Inception V4 [9] propose une version rationalisée, à l'architecture uniforme et aux performances accrues.

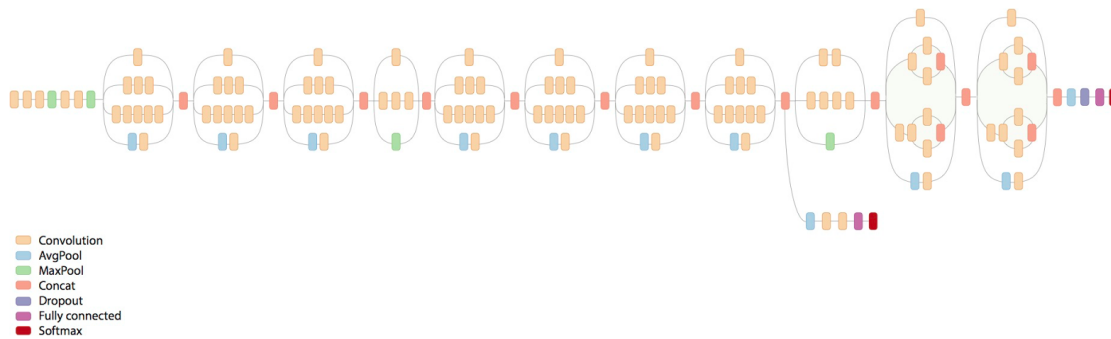


FIGURE 1-6 – Architecture d'inception V3

1.4- ResNet

En 2015, Microsoft remporte la compétition ILSVRC avec ResNet [3], un réseau à 152 couches qui utilise un module ResNet. Le taux de bonne reconnaissance est de 96.4%. Un réseau résiduel (ou ResNet) résout le problème de vanishing gradient de la manière la plus simple possible, en permettant des raccourcis entre chaque couche du réseau. Dans un réseau classique, l'activation en sortie de couche est de la forme $y = \sigma(x)$, et lors de la rétropropagation, le gradient doit nécessairement repasser par $\sigma(x)$, ce qui peut causer des problèmes en raison de la (forte) non linéarité induite par σ . Dans un réseau résiduel, la sortie de chaque couche est calculée par $y = \sigma() + x$, où $+x$ est le raccourci entre chaque couche, qui permet au gradient de transiter directement sans passer par σ .

Cette représentation donne l'idée générale, mais la réalité est un peu plus complexe, et prend la forme d'un module ResNet (figure 1-7).

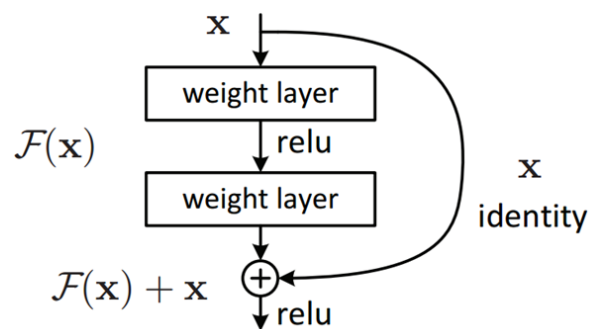


FIGURE 1-7 – Module ResNet (source : [3])

1.5- SqueezeNet

SqueezeNet [4] est un réseau produit en 2016, qui n'est pas tant remarquable par ses performances (il atteint les mêmes niveaux de reconnaissance qu'AlexNet), mais par sa légèreté (le modèle entraîné sur ImageNet a une taille de 4.9 Mo, et possède 50 fois moins de paramètres qu'AlexNet par exemple) et la rapidité avec laquelle il peut être entraîné.

SqueezeNet introduit des modules "Fire" (figure 1-8), composés d'une couche de convolution "squeeze",

dotée de filtres de taille 1×1 , et d'une couche d'expansion dotée de filtres de taille 1×1 et 3×3 . L'utilisation de filtres 1×1 permet une réduction du nombre de paramètres.

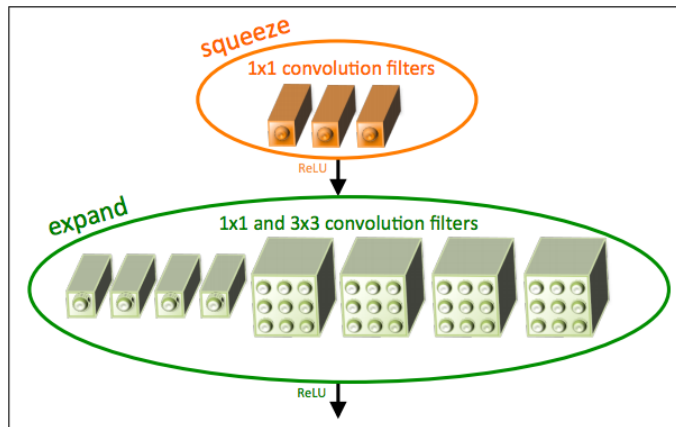


FIGURE 1-8 – Module Fire (source : [4])

Le réseau est composé d'une couche de convolution classique, d'une couche d'agrégation max, suivie de 9 modules Fires entrecoupées d'agrégation max, et d'une couche de convolution finale. Le nombre de filtres est progressivement augmenté entre chaque module (figure 1-9).

layer name/type	output size	filter size / stride (if not a fire layer)	depth	$s_{1 \times 1}$ (#1x1 squeeze)	$e_{1 \times 1}$ (#1x1 expand)	$e_{3 \times 3}$ (#3x3 expand)	$s_{1 \times 1}$ sparsity	$e_{1 \times 1}$ sparsity	$e_{3 \times 3}$ sparsity	# bits	#parameter before pruning	#parameter after pruning
input image	224x224x3										-	-
conv1	111x111x96	7x7/2 (x96)	1				100% (7x7)			6bit	14,208	14,208
maxpool1	55x55x96	3x3/2	0									
fire2	55x55x128		2	16	64	64	100%	100%	33%	6bit	11,920	5,746
fire3	55x55x128		2	16	64	64	100%	100%	33%	6bit	12,432	6,258
fire4	55x55x256		2	32	128	128	100%	100%	33%	6bit	45,344	20,646
maxpool4	27x27x256	3x3/2	0									
fire5	27x27x256		2	32	128	128	100%	100%	33%	6bit	49,440	24,742
fire6	27x27x384		2	48	192	192	100%	50%	33%	6bit	104,880	44,700
fire7	27x27x384		2	48	192	192	50%	100%	33%	6bit	111,024	46,236
fire8	27x27x512		2	64	256	256	100%	50%	33%	6bit	188,992	77,581
maxpool8	13x12x512	3x3/2	0									
fire9	13x13x512		2	64	256	256	50%	100%	30%	6bit	197,184	77,581
conv10	13x13x1000	1x1/1 (x1000)	1				20% (3x3)			6bit	513,000	103,400
avgpool10	1x1x1000	13x13/1	0									
activations parameters compression info											1,248,424 (total)	421,098 (total)

FIGURE 1-9 – Architecture de SqueezeNet (source : [4])

2- COMMENT UTILISER CES RÉSEAUX

Il est possible de redéclarer les réseaux classiques depuis TensorFlow ou Keras, en définissant une à une les couches et leur paramètres, qui sont décrits dans les articles correspondants. On imagine assez bien le travail que cela peut représenter sur le ResNet de Microsoft...

Fort heureusement, il existe d'autres manières d'utiliser ces réseaux.

2.1- Utilisation de réseaux pré-entraînés

Il est possible avec TensorFlow et Keras de charger / sauvegarder des réseaux qui ont été entraînés. Il est également possible, pendant l'entraînement, de créer des sauvegardes (checkpoints) pour reprendre éventuellement l'entraînement en cours d'itérations. On peut sauvegarder tout le réseau (architecture + optimiseur + poids), ou seulement les poids.

Pour charger / sauver les poids d'un modèle `model` Keras, on utilise simplement les fonctions

```
model = create\_model()\nmodel.load\_weights('./checkpoints/my\_checkpoint')
```

et

```
model.save\_weights('./checkpoints/my\_checkpoint')
```

Pour charger / sauver le modèle complet, on utilise

```
model = keras.models.load\_model('my\_model.h5')
```

et

```
model = create\_model()\nmodel.fit(xtrain, ytrain)\nmodel.save('my\_model.h5')
```

La procédure sous Tensorflow est un peu plus complexe et est décrite [ici](#).

2.2- Transfer learning et fine tuning

Il est possible d'utiliser les réseaux classiques pré-entraînés pour de nouvelles tâches. L'idée sous-jacente est que les premières couches capturent des caractéristiques bas niveau, et que la sémantique vient avec les couches profondes. Ainsi, dans un problème de classification, où les classes n'ont pas été apprises, on peut supposer qu'en conservant les premières couches on extraira des caractéristiques communes des images, et qu'en changeant les dernières couches (information sémantique et haut niveau et étage de classification), c'est à dire en réapprenant les connexions, on spécifiera le nouveau réseau pour la nouvelle tâche de classification.

Cette approche rentre dans le cadre des méthodes de Transfer Learning [7] et de fine tuning, cas particulier d'adaptation de domaine :

- les méthodes de transfer learning prennent un réseau déjà entraîné, enlèvent la dernière couche complètement connectée, et traitent le réseau restant comme un extracteur de caractéristiques. Un nouveau classifieur est alors entraîné sur les caractéristiques calculées sur le nouveau problème
- les méthodes de fine tuning ré-entraînent le classifieur du réseau, et remettent à jour les poids du réseau pré-entraîné par rétropropagation.

Plusieurs facteurs influent sur le choix de la méthode à utiliser : la taille des données d'apprentissage du nouveau problème, et la ressemblance du nouveau jeu de données avec celui qui a servi à entraîner le réseau initial :

- pour un jeu de données similaire de petite taille, on utilise du transfer learning, avec un classifieur utilisé sur les caractéristiques calculées sur les dernières couches du réseau initial
- pour un jeu de données de petite taille et un problème différent, on utilise du transfer learning, avec un classifieur utilisé sur les caractéristiques calculées sur les premières couches du réseau initial
- pour un jeu de données, similaire ou non de grande taille, on utilise le fine tuning

A noter qu'il est toujours possible d'augmenter la taille du jeu de données par des techniques de "Data Augmentation" (changement de couleurs des pixels, rotations, cropping, homothéties, translations...)

3- QUE FAIRE SI J'AI PEU DE DONNÉES ?

Les méthodes supervisées nécessitent pour de bonnes performances un ensemble d'apprentissage S de grand cardinal. Si seulement peu d'exemples $S_s = \{(\mathbf{x}_i, y_i), i \in [1 \dots m]\}$ sont disponibles, avec m petit, les

techniques précédemment décrites ne sont pour la plupart plus applicables.

Les méthodes de Few-Shot Learning ont été introduites pour traiter ce manque de données. Les exemples applicatifs sont nombreux, allant de la classification d'images à l'analyse de sentiments à partir de textes, ou encore à la reconnaissance d'objets.

Vu sous l'angle de la minimisation du risque empirique, l'hypothèse h construite sur la minimisation de

$$R(h) = \sum_{i=1}^m \ell(y_i, h(\mathbf{x}_i))$$

conduit à un sur apprentissage et un risque $R(h)$ très loin du risque réel. Pour pallier ce problème, des connaissances *a priori* doivent être utilisées. Le Few-shot learning propose trois alternatives. Nous détaillons ici l'une d'entre elles, l'augmentation de données.

Les approches de cette catégorie utilisent des connaissances *a priori* sur les données pour enrichir $\mathcal{S}_s$. On les regroupe parfois sous le vocable de méthodes d'augmentation de données. Si elles sont faciles à mettre en oeuvre et à comprendre, ces méthodes restent cependant dépendantes du domaine d'étude et ne peuvent être facilement généralisées.

Les principales stratégies sont résumées dans le tableau .1 et un exemple d'illustration est donné figure 3-10.

Transformation...	Entrée	Opérateur	Sortie
... de données de $\mathcal{S}_s$	$(\mathbf{x}_i, y_i) \in \mathcal{S}_s$	$t : \mathcal{X} \rightarrow \mathcal{X}$	$(t(\mathbf{x}_i), y_i)$
... d'un ensemble de données non étiquetées	$(\mathbf{x}, -)$	$h : \mathcal{X} \rightarrow \mathcal{Y}$ entraîné sur $\mathcal{S}_s$	$(\mathbf{x}, h(\mathbf{x}))$
... d'un ensemble de données similaires	$\{(\hat{\mathbf{x}}_j, \hat{y}_j)\}$	Opérateur de combinaison c	$(c(\{\hat{\mathbf{x}}_j\}), c(\{\hat{y}_j\}))$

TABLE .1 – Techniques d'augmentation de données



FIGURE 3-10 – Exemple d'augmentation de données. De gauche à droite : image originale, rotation de 20°, flip, ajout de bruit gaussien, déformation élastique, changement de contraste par canal RGB.

4- PARTIE PRATIQUE

Vous avez à disposition un notebook vous proposant d'apprendre, à partir d'un réseau pré-entraîné ([MobileNetV2](#), assez léger et performant en classification), de fine-tuner ce dernier pour un problème où les données ne font pas partie de la base d'entraînement initiale (l'objectif ici est d'apprendre à classifier des fleurs en 5 catégories). Votre travail consiste à :

- Appliquer une technique d'augmentation de données pour augmenter le nombre d'images de fleurs. Pour ce faire, vous utiliserez l'objet [ImageDataGenerator](#), que nous vous laissons prendre en main...
- Créer le modèle. vous chargerez tout d'abord le réseau MobileNetV2 (fonction [MobileNetV2](#)). Vous supprimerez ensuite la dernière couche de ce réseau (la couche de classification) à l'aide de la fonction [pop](#), puis ajouterez votre propre couche de classification. Vous indiquerez ensuite quels poids ne doivent pas être réappris (en taggant les couches correspondantes avec le drapeau `layer.trainable = False`), et réentraînez le nouveau réseau sur la nouvelle base augmentée. Vous pourrez tester l'influence du nombre de couches non réentraînées.
- Evaluer la précision de votre réseau.

- [1] Alfredo Canziani, Adam Paszke, and Eugenio Culurciello. An analysis of deep neural network models for practical applications. CoRR, abs/1605.07678, 2016.
- [2] François Chollet. Xception : Deep learning with depthwise separable convolutions. CoRR, abs/1610.02357, 2016.
- [3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. CoRR, abs/1512.03385, 2015.
- [4] Forrest N. Iandola, Matthew W. Moskewicz, Khalid Ashraf, Song Han, William J. Dally, and Kurt Keutzer. Squeezenet : Alexnet-level accuracy with 50x fewer parameters and <1mb model size. CoRR, abs/1602.07360, 2016.
- [5] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, page 2012, 2012.
- [6] Min Lin, Qiang Chen, and Shuicheng Yan. Network in network. CoRR, abs/1312.4400, 2013.
- [7] Sinno Jialin Pan and Qiang Yang. A survey on transfer learning. *IEEE Trans. on Knowl. and Data Eng.*, 22(10) :1345–1359, October 2010.
- [8] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. CoRR, abs/1409.1556, 2014.
- [9] Christian Szegedy, Sergey Ioffe, and Vincent Vanhoucke. Inception-v4, inception-resnet and the impact of residual connections on learning. CoRR, abs/1602.07261, 2016.
- [10] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott E. Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. CoRR, abs/1409.4842, 2014.
- [11] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. CoRR, abs/1512.00567, 2015.